

A Provably Secure True Random Number Generator with Built-In Tolerance to Active Attacks

Berk Sunar, *Member, IEEE Computer Society*, William J. Martin, and Douglas R. Stinson

Abstract—This paper is a contribution to the theory of true random number generators based on sampling phase jitter in oscillator rings. After discussing several misconceptions and apparently insurmountable obstacles, we propose a general model which, under mild assumptions, will generate provably random bits with some tolerance to adversarial manipulation and running in the megabit-per-second range. A key idea throughout the paper is the fill rate, which measures the fraction of the time domain in which the analog output signal is arguably random. Our study shows that an exponential increase in the number of oscillators is required to obtain a constant factor improvement in the fill rate. Yet, we overcome this problem by introducing a postprocessing step which consists of an application of an appropriate resilient function. These allow the designer to extract random samples only from a signal with only moderate fill rate and, therefore, many fewer oscillators than in other designs. Last, we develop fault-attack models and we employ the properties of resilient functions to withstand such attacks. All of our analysis is based on rigorous methods, enabling us to develop a framework in which we accurately quantify the performance and the degree of resilience of the design.

Index Terms—True (and pseudo) random number generators, resilient functions, cryptography.



1 INTRODUCTION

RANDOM number generators have numerous applications in a diverse set of areas ranging from statistics to cryptography to art. For many applications, pseudorandom number generators (PRNGs)—which take a short random string and expand it into a stream of “random looking” bits using a deterministic algorithm—are quite satisfactory. However, for cryptographic applications, it is crucial to generate pseudorandom bits which will be unpredictable even by the strongest adversary. Furthermore, even if high quality PRNGs, with their output being computationally indistinguishable from the output of a true random number generator (TRNG), may be built, these PRNGs will still need to be seeded using TRNGs.

Good TRNG design rests on the quality of three components:

- **Entropy Source:** Various TRNG designs have been proposed for harvesting the randomness present in physical processes such as thermal and shot noise in circuits, brownian motion, or nuclear decay. The entropy source is the most critical component as it determines the available entropy. Some sources exhibit biases; these should be eliminated in the collection or postprocessing steps.
- **Harvesting Mechanism:** The entropy source is tapped using a harvesting mechanism that does

not disturb the physical process above, yet “collects” as much entropy as possible. Various designs have been proposed to realize this step. A decent harvesting mechanism should come with rigorous justification with the underlying assumptions regarding the source explicitly stated.

- **Postprocessing:** Although this component is not needed in all designs, many TRNG designers strengthen their designs by postprocessing the output bits. A postprocessor may be applied to mask imperfections in the entropy source or harvesting mechanism or to provide tolerance in the presence of environmental changes and tampering. A postprocessor may be as simple as a von Neumann corrector [4] or may be as complicated as an extractor function [1] or a one-way hash function such as SHA-1 [4]. One should scrutinize postprocessors which modify the output conditionally on its statistical properties. There is great danger in deterministic methods aimed at improving the “appearance of randomness.”

From a practical standpoint, it is essential that TRNGs be built using a commonly available cheap silicon process. Moreover, it is highly desirable to implement TRNGs using purely digital design technique. This allows for easier integration with digital microprocessors and also makes it possible to implement TRNGs on popular reconfigurable platforms (i.e., FPGAs and CPLDs). To date, various random number generator designs have been proposed based on mixed or pure digital electronics. These designs vary significantly according to their entropy sources and the harvesting techniques they employ. For instance, the design introduced in [6] uses a combination of analog and digital components for amplification and sampling of white noise.

The main problem with this type of circuit is the amplification

• B. Sunar is with the Worcester Polytechnic Institute, Atwater Kent Room 320, 100 Institute Rd., Worcester, MA 01609. E-mail: sunar@wpi.edu.
 • W.J. Martin is with the Worcester Polytechnic Institute, Stratton Hall 109B, 100 Institute Rd., Worcester, MA 01609. E-mail: martin@wpi.edu.
 • D.R. Stinson is with the David R. Cheriton School of Computer Science, University of Waterloo, Waterloo, Ontario, N2L 3G1, Canada. E-mail: dstinson@uwaterloo.ca.

Manuscript received 10 June 2005; revised 31 Mar. 2006; accepted 7 July 2006; published online 22 Nov. 2006.

For information on obtaining reprints of this article, please send e-mail to: tc@computer.org, and reference IEEECS Log Number TC-0194-0605.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on June 04, 2026 at 09:27:58 UTC from IEEE Xplore. Restrictions apply.

stage, which requires significant power to bring the noise level up a few orders of magnitude to the digital logic level. A similar design was developed by Intel Corp. [4], where the thermal noise on a junction is amplified and used to drive a voltage-controlled oscillator which is then sampled by another oscillator. The output sequence is postprocessed using the von Neumann corrector and SHA-1. The design in [7] samples the jitter in a phase-locked loop (PLL)—an analog component—on a specialized reconfigurable logic platform. The innovative design introduced in [8] randomly samples the output of an LFSR and a cellular automaton. The randomness comes from the jitter in the two oscillator circuits which are used to clock the two deterministic circuits. Although the design is purely digital and no amplification is needed, it is difficult to verify the harvesting/sampling technique due to a complicated harvesting scheme. Moreover, despite the size of the design, the entropy source is limited to the two oscillators. In another design [11], a simple architecture based on metastable circuits is proposed. The design passes the statistical tests only when a large number of such circuits are combined.

Indeed, all of these designs are validated by running the DIEHARD [12] or NIST Test Suites [13]. Typically, the optimal sampling frequency is determined by trial and error: The sampling frequency is decreased until the output sequence starts to pass the NIST or DIEHARD tests. Furthermore, due to the complex harvesting schemes employed, it is very difficult to give a mathematical justification for proper collection of the entropy. Due to the shortcomings of PRNG tests applied to true random number generators, new tests and standards are needed. Recently, Schindler and Killman [18] sketched a methodology for evaluating true random number generators and outlined the pioneering standardization efforts of the German Department for Security in Information Technology BSI as described in [20]. In their treatment, Schindler and Killman advocate rigorous testing of TRNGs and note that a statistical blackbox testing strategy may not be employed for this purpose. The AIS document provides clear evaluation criteria for TRNGs and also allows TRNG designers to present their own criteria. In [16], Dichtl presents an attack on a TRNG construction underlining the importance of providing rigorous security proofs or justifications for TRNG designs. In a more recent work, Bucci and Luzzi [19] emphasize the importance of designing for testability and make the observation that it is difficult and perhaps impossible to test the quality of TRNGs after complex postprocessing techniques have been employed. Hence, the authors propose the use of stateless TRNG circuits which allow tests to be formulated. Furthermore, the authors of the same reference note that robustness against attacks, faults, and production defects should also be considered as factors when considering the quality of a TRNG design.

Inspired by these works and observations, we develop a set of requirements for our TRNG design:

- The design should be purely digital (no analog components are allowed).
- The harvesting mechanism should be simple (easy to analyze); it should preserve and optimally sample the

entropy source. In other words, the unpredictability of the TRNG should be based not on the complexity of the harvesting mechanism, but only on the unpredictability of the entropy source.

- A strict mathematical justification of the entropy collection mechanism should be given, with all assumptions clearly stated and at least empirically justified. The design should be sufficiently simple to allow rigorous analysis.
- No correction circuits are allowed. Many times an adaptive correction circuit is used either to “adjust the sampling frequency” or to “smooth the output distribution.” Since most such circuits use the characteristic of the output to adaptively process the entropy source, they introduce further correlations. For instance, a correction circuit that counts the number of ones and zeros and compensates for the delay of a sampler accordingly will clearly introduce further bias to the output sequence.
- Compact and efficient design (high throughput per area and energy spent). No amplifiers or other analog components are allowed, which would consume more energy and make the analysis difficult. Note that, since we are not allowing analog components, we have to sample variations in the time domain (such as the design in [8] does) rather than variations in the voltage levels. This criterion also means that we cannot use complicated post-processing schemes (e.g., SHA-1).

We propose a design that meets all of these requirements and allows for fault-tolerant extensions.

In the remainder of this paper, we introduce a TRNG design that uses jitter (or random vibration) in clock signals present in all digital clocked circuits. Since such a signal will be available in most applications, this approach is particularly attractive. We outline several harvesting techniques and introduce a sampling method based on resilient functions.

2 OSCILLATOR RINGS

Oscillators provide a simple and effective method to build TRNGs [5]. A simple digital oscillator may be built by chaining an odd number of inverter gates in a ring configuration. Due to the feedback path, the output of any of the inverters will oscillate from a logic one to a logic zero and back. Hence, a square wave signal is obtained by tapping into the oscillator at any point in the ring. The signal has several characteristics:

- Ideally, the output signal Ψ is a periodic square wave with its period determined by the number of inverters and the delay of an inverter, i.e., $T = n \cdot \tau$ and $\Psi(t) = \Psi(t + T)$, where T denotes the period and τ denotes the delay of a single inverter.
- The output signal is *not* a perfect square wave. For instance, the period vibrates in a random manner $T = T + \hat{T}$, where $\hat{T}$ represents a random variable that takes values in the range $(-T/2, T/2)$. In a high quality digital circuit, the range of $\hat{T}$ is quite small compared to the period. The vibration in the clock

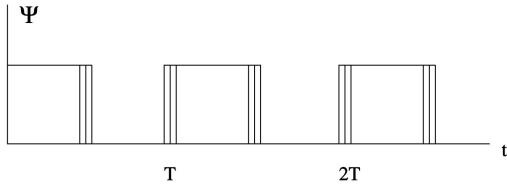


Fig. 1. Periodic oscillator signal with transition zones marked as multiple lines.

signal, denoted by the random variable $\hat{T}$, is commonly called *jitter*. Jitter is the source of entropy we wish to harvest.

In Fig. 1, a typical oscillator output is depicted. The jitter is represented via extra lines at each transition of the waveform.

A practical configuration for harvesting jitter is based on the idea of sampling the output of a ring oscillator using the output of another oscillator. This configuration is commonly referred to as *coupled oscillators*. If the periods of the two oscillators are well matched, then it should be that, with high probability, we are sampling from the transition zones and not the deterministic part of the waveform. Unfortunately, problems arise in practical realization of coupled oscillators:

- Exactly matching the period of the two oscillators is fairly difficult and requires the use of special layout design techniques at the VLSI level.
- Due to imperfections, the two signals may *drift* relative to one another. This makes for very fragile TRNG designs.

To alleviate these two problems, additional circuitry (e.g., a compensator) is sometimes used to adjust the waveforms to match the transitions. This solution, however, *corrupts* the randomness of the jitter and introduces biases. In the next section, we consider more sophisticated methods for combining and sampling oscillators.

3 COMBINING SIGNALS TO EXPLOIT RANDOMNESS OF PHASE JITTER

We have r distinct oscillator rings arranged on a chip design as illustrated in Fig. 2. The i th ring is composed of n_i inverters, where the optimal parameters n_i and r are to be determined below. The analog output signal Ψ of the circuit is the XOR of the outputs Ψ_j of the r individual rings $R_1, \dots, R_r$. The XOR function is typically implemented as a

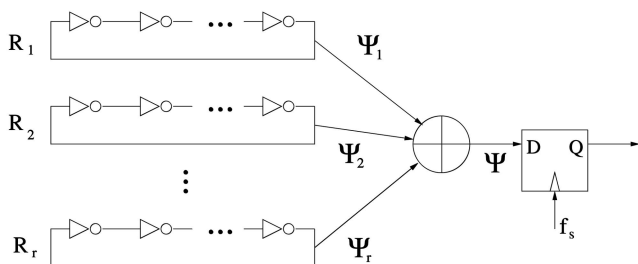


Fig. 2. Design for combining and sampling oscillators.

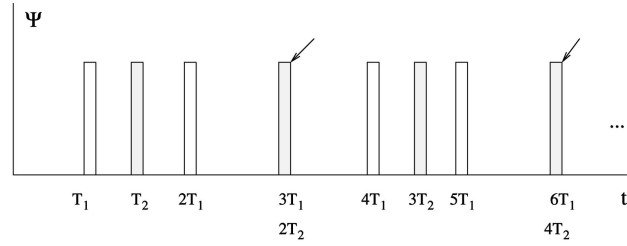


Fig. 3. Output waveform of the XOR of two oscillator outputs with periods T_1 and T_2 .

binary XOR-tree. The output Ψ is converted into a digital signal by sampling it at a regular clock frequency f_s .

The output of the XOR combines periodic transition zones contributed by each ring. Since the entire waveforms are XORed, the output will also have deterministic regions and, occasionally, zones where jitter contributed from different rings overlaps. The resulting waveform exhibiting this effect with only two rings is depicted in Fig. 3. The deterministic parts of the waveform are not shown for clarity. The two oscillators are on identical phase and their ideal periods are related by $2T_2 = 3T_1$. Both oscillators contribute a transition zone at $t = 2T_2$ and any other even multiple of T_2 (marked by arrows). Our purpose is to populate the entire spectrum with such transition zones and then sample the waveform only in such zones. Due to such overlaps, we lose out on the entropy. Later, we will tackle this problem, introducing several approaches to its resolution.

4 THE URN MODEL—A COMBINATORIAL APPROACH

The combined signal is sampled to provide a random bitstream. The sampling of this combined signal Ψ is modeled as follows: The behavior of the XOR gate on signals which lie strictly between zero and one is well-documented to behave as a smooth interpolation. The combined signal is assumed to lie between L and H volts, where $L < H$. A signal value closer to L is deemed to give rise to a zero bit and a signal value closer to H volts is recorded as a one.

We take the following axiomatic approach, which, for the time being, neglects phase drift. Suppose R_j is an oscillator ring with period $T = T_j$. That is, at times $0, T, 2T, \dots$, the signal is designed to switch from low to high and, at times $T/2, 3T/2, 5T/2, \dots$, the signal is designed to switch from high to low. (Note that we aim to harvest jitter only from the “up” transition zones, allowing a full period before sampling jitter from the same ring.)

Assumption. In any open time interval $(mT - T/4, mT + T/4)$, there is a unique point t where the signal crosses $(L + H)/2$ volts and this t behaves as a normally distributed random variable with mean mT and some variance σ_j^2 .

This hypothesis will be the basis of our claims regarding the random behavior of the overall system. Of course, such an assumption is unavoidable as any device which purports to produce random bits must be assumed to have access to some random physical source. Fortunately, the claim in our

axiom is strongly supported by empirical evidence. In [2], for example, over a million oscilloscope captures (sampling at 4 Giga samples/sec) of jitter from a single oscillator ring with 83 inverters having $T \approx 146.8\text{ns}$ ($f = 6.81\text{ MHz}$) were displayed and exhibited classical bell curve behavior.

Consider a time interval $I = [a, b]$ such as $[100T, 1, 100T]$ and suppose, for the sake of simplicity, that our threshold voltage is $(L + H)/2 = 2.5V$. Here, L and H stand for the value of a voltage that is accepted by a particular technology as logic 0 and logic 1, respectively, e.g., L is 0 V or GROUND and H is 5 V. Our goal is to “fill up this interval with randomness” using our combined signal composed of the output of k oscillator rings. More precisely, we would like to ensure that, for some threshold p , at any point t in interval I , there exists some ring R_j such that $1/4 < \text{Prob}[\Psi_j < 2.5] < 3/4$.¹

Appealing to standard probability tables, it therefore suffices to require that, for some integer m , $|t - mT_j| < 0.6745\sigma_j$. (More explicitly, the area under the normal distribution curve from $\mu - 0.6745\sigma$ to $\mu + 0.6745\sigma$ gives the desired 0.5 probability, where μ represents the mean and σ the standard deviation.) We will later relax this condition by simply asking that it hold with at least some probability q (e.g., $q = 0.9$) when t is chosen uniformly at random from interval I :

Criterion A. *With t chosen uniformly at random from the interval I , the probability that there exist integers j and m with $|t - mT_j| < 0.6745\sigma_j$ is at least q .*

In an ideal model, the above design criterion is not even sufficient to ensure that the output bitstream will be balanced. Indeed, if the circuit consists of a single oscillator ring and the clock is perfectly calibrated to sample at times $mT + \epsilon$, then the above model holds, but the output will consist almost entirely of ones. While the model can be refined to eliminate the possibility of such bias, we will work with it as given with the knowledge that phase drift and the large number of rings used makes such biased behavior highly improbable.

An extremely useful simplification in this regard is to reduce the problem to a combinatorial one. Let us consider a long time interval I and partition it into small equal-length subintervals in such a way that Criterion A is satisfied (with $q = 1$) for any time t in a subinterval J provided some ring R_j is in transition at some point t' in the same subinterval. That is, we discretize the time domain in such a way that any jitter event occurring in any subinterval J makes Criterion A hold (with $q = 1$) for all times t in interval J . We will call these subintervals “urns” because, as we shall soon see, the problem which now faces us is one well-known to probabilists who study urn models.

Here are the precise details of how the interval is to be subdivided into “urns.” Let the interval $I = [a, b]$ be subdivided into ℓ equal length subintervals $J_1, \dots, J_\ell$, where $J_h = [a + (h-1)\frac{b-a}{\ell}, a + h\frac{b-a}{\ell}]$. For the purpose of this discussion, we will refer to each J_h as an *urn*. An urn J

is to be considered full if there is some ring R_j in the circuit whose signal satisfies $\Psi_j(t) = 2.5$ for some real number t in the urn J . Otherwise, the urn is empty.

Now we see that Criterion A is satisfied provided

- at least $q\ell$ of the urns are filled and
- $\ell > (b-a)/0.6745\sigma_j$ for all $j = 1, \dots, k$.

Ignoring phase drift, we see that ring R_j will fill roughly one out of every $\pi_j = \ell T_j / (b-a)$ urns. (We call π_j the “combinatorial period” of the ring R_j .) The next step is to decide how many inverters each ring should have in order to efficiently populate such an interval I with jitter events.

Note that we are not concerned by multiple occupancies—i.e., multiple rings contributing to a single urn—since the entropy of an exclusive-OR is always at least the entropy of any of the inputs. Thus, multiple occupancies can only benefit our model.

5 RELATIVELY PRIME RING LENGTHS

Looking at Fig. 3, one notices that there is “waste” of entropy when two rings are in transition at the same point in time. One is therefore tempted to find ways to minimize such overlap. It has been proposed that, in order to fill as many urns as possible and in order to make the behavior of the r rings as independent as possible, the ring lengths $n_1, \dots, n_r$ should be pairwise relatively prime integers. While one experiment with such a design produced a digital output with good randomness properties (according to the NIST tests), the randomness is apparently not applicable to the above model. We now show this using the Chinese Remainder Theorem.

Suppose that ring R_j fills urns $d_j + m\pi_j$ ($m \geq 0$). It is reasonable to assume that, if the ring lengths n_j are pairwise relatively prime, then the combinatorial periods π_j are close to relatively prime as well. Then, with $N = \pi_1\pi_2 \cdots \pi_r$, we have an urn J_h filled if there exists a j with $h \equiv d_j \pmod{\pi_j}$. The urn is therefore empty whenever $h \not\equiv d_j \pmod{\pi_j}$ for all $j = 1, \dots, r$. Modulo period π_j , there are $\pi_j - 1$ values not equal to d_j . So, an integer h with $0 \leq h < N$ represents an unfilled urn whenever $h \equiv e_j \pmod{\pi_j}$ for all j unless some $e_j = d_j$ modulo π_j . For any such system of congruences, there is a unique solution h modulo N . So, among the N urns, $\prod_{j=1}^r (\pi_j - 1)$ will be left unfilled. In other words, a randomly chosen urn will be left unfilled with probability

$$\left(1 - \frac{1}{\pi_1}\right) \left(1 - \frac{1}{\pi_2}\right) \cdots \left(1 - \frac{1}{\pi_r}\right),$$

requiring, for example, over 800 rings when all π_j are distinct primes with $\pi_j \geq 25$ and with a fill rate of $2/3$. Moreover, the entire motivation for the relatively prime ring length model is unclear. Under our axiomatic assumption, the location of the transition point for the j th ring is a normally distributed random variable. Ignoring phase drift, it carries no memory; there is no correlation between consecutive bits influenced by this single ring. Thus, this feature of the model is not only impractical, but its value to the model is questionable to begin with.

1. The tolerance of $1/4$ was chosen arbitrarily. Note that a smaller tolerance gives us more confidence that a sample near this point will behave like an unbiased coin flip. A smaller tolerance increases the reliability of the design, but also decreases the efficiency, as we will see shortly.

TABLE 1
Urn Width for Various Levels of Entropy per Output Bit

Target entropy	$p = H_2^{-1}(E)$ min $Pr[X = 0]$	$A^{-1}(p)$ tolerance	num. urns ($w = 0.02T$)
0.99	0.441198	0.145σ	173
0.97	0.398388	0.258σ	97
0.95	0.269128	0.335σ	75
0.90	0.316019	0.479σ	52
0.80	0.243004	0.699σ	36
0.50	0.110028	1.229σ	20

6 IDENTICAL RING LENGTHS

Our proposal is to take advantage of the random delays d_j as well as the phase drift of the signals relative to one another by making all ring lengths identical. Indeed, it is possible that two rings with an equal number of inverters will exhibit a great deal of overlap in their transition zones. These nondeterministic elements (initial delay and phase drift) make it highly likely that two such rings will have *no* overlap in their contributions to the jitter.

In what follows, we will show that, using identical ring lengths, we can achieve reasonable fill rates with only a moderate number of rings, thereby keeping the total number of inverters down and allowing for portability of the hardware to fairly compact platforms.

Based on our combinatorial urn model, the problem of filling a long time interval I with jitter using identical ring lengths becomes one quite familiar to probabilists. This is the so-called “Coupon Collector’s Problem” and it has been well-studied in discrete probability.

The Coupon Collector’s Problem falls under the category of “urn models” in probability and is given as follows: There are N different coupons, but an unlimited supply of each of these. A coupon collector collects one coupon per day, choosing a coupon from among the N uniformly at random, with replacement.² The challenge is to compute the expected number of days, r , which will pass before the collector has at least one copy of each of the N distinct coupons. We will need to delve deeper and estimate the number of days before the collector has 80 percent confidence that she has at least 95 percent of the coupons or, more generally, confidence c out of 100 that the current number of coupons in her possession is at least fN , where $0 < f \leq 1$.

6.1 Urn Width

Before we get into the combinatorics of the coupon collector’s problem, we need to know a bit about the actual numbers involved. A crucial parameter for the design is the width of these intervals we are calling urns. Suppose we have a design where each ring has a fixed number n of inverters. This determines a period T for the ring and, depending on the specifics of the hardware, a “jitter width” w , which is technically the standard deviation of the associated random variable. Based on our desired entropy per bit of output, our urn width may differ from this value. Table 1 illustrates the urn width for various

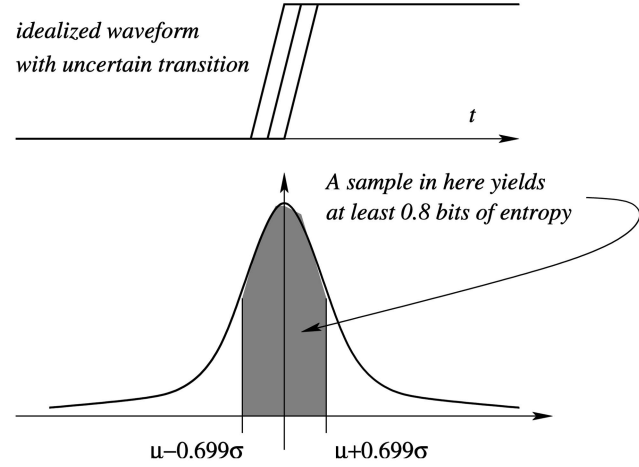


Fig. 4. A single jitter event; sampling close to the mean yields a bit with high entropy.

levels of entropy per output bit. For instance, for a target entropy of 0.80, the sample must be collected within the interval of $(\mu - 0.699\sigma, \mu + 0.699\sigma)$, as shown in Fig. 4.

For a target entropy E per sampled bit, we take the inverse of the binary entropy function $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ to find how close to 0.5 the probability of sampling a zero must lie. The column $A^{-1}(p)$ is half the desired width of our urn.³ If sampling occurs within the interval $(\mu - A^{-1}(p), \mu + A^{-1}(p))$, then we are guaranteed the prescribed entropy per sampled bit. So, the number of urns (using the example where the standard deviation w is two percent of the period), is given by $50/2A^{-1}(p)$. For other ratios of standard deviation to period, $\omega = \sigma/T$, we simply multiply the above number of urns by $0.2/\omega$.

6.2 Phase Drift

Whereas, in the model involving relatively prime ring lengths, phase drift of one component relative to another constitutes a major complication, phase drift can only improve the randomness of the identical ring length model. Indeed, in each sequence of π consecutive urns, we are assuming that each ring fills one randomly chosen urn (that urn whose index is congruent to d_j modulo π) and that these d_j are independently chosen uniformly at random. Experimental evidence suggests exactly that phase drift, if it does occur in any nonnegligible way, is independent from one ring to another [3]. In our model, this is simply interpreted as ring R_j choosing a slightly different delay d_j in each interval I of π urns. The delays remain uniformly distributed and independently chosen, so the model still applies.

The only complication is that, in the event that some ring has its transition point close to the end of I (i.e., that d_j is close to zero modulo π), a ring may fill no urns or two urns in some consecutive sequence of N urns. This can be compensated for by making a slight increase in the total number of rings so that in any such sequence of N consecutive urns, the number of rings filling at least one urn is at least the desired number in the above table with very high probability.

2. Since two transition zones may overlap, our model must allow more than one ring to fill the same urn. This is why filled urns are “replaced.”

3. Here, $A(z)$ denotes the area under the normal density function (the bell curve) between μ and z .

TABLE 2
The Expected Number of Rings Required to Fill fN Urns
for Various Values of the Fill Rate f

N	f					
	0.50	0.60	0.70	0.80	0.90	1.00
50	34.2	45.1	59.0	78.5	110.8	224.9
100	68.8	90.9	119.2	159.0	225.8	518.7
200	138.1	182.5	239.6	319.9	456.0	1175.6

6.3 Coupon Collecting

With all ring lengths identical, the time spectrum gets partitioned into equal-length subintervals, which we call “urns.” If we focus on the urns within one period of the ring, we wish to fill as many of these as possible using as few rings as possible. Allowing for the randomness of phase shifts and owing to uncontrollable phase drift, we model each oscillator ring as filling one urn per period with a jitter event in such a way that sampling at any point in this subinterval is assumed to yield a random bit.

Our problem can therefore be modeled as one in discrete probability: the coupon collector problem, where one selects one urn from a set of N uniformly at random until each urn has been “filled” or selected at least once. The power and limitation of this approach is already evident from the most elementary analysis of the coupon collector problem.

Let us add rings to the design one at a time and view these as uniformly random selections of one urn from among N . Suppose that, at some point in time, all but a fraction qN of the N urns have been filled. Then, there is a probability q that the next selection will fill a new urn and a probability $(1-q)^{k-1}q$ that it will require exactly k more selections before a new urn is filled. So, the expected number of selections required to fill the next urn is

$$\sum_{k=1}^{\infty} (1-q)^{k-1} \cdot q \cdot k = \frac{1}{q}.$$

For example, when half the urns are filled, the expected number of additional rings required to fill the next one is two and, when 3/4 are filled, it is four.

Therefore, the expected number of rings needed to fill all N urns is

$$r = \sum_{s=1}^N \frac{N}{s} = N \sum_{s=1}^N \frac{1}{s} \approx N \log N, \quad (1)$$

where $\log(\cdot)$ denotes the natural logarithm function.

Clearly, this suggests that we need a very large number of rings. Of course, we are penalized the most for our desire to fill even the last few urns. For example, with $N = 50, 100$, and 200 , Table 2 gives the expected number of rings required to fill fN urns for various values of f . The table is built by computing the exact summation using (1). For instance, to compute the number of rings r needed for $N = 100$ urns to obtain a fill rate of $f = 0.70$, we use the summation $r = 100 \cdot \sum_{s=31}^{100} 1/s$.

So, our strategy, of course, is to aim for a fill rate, f , less than 1.0 and to compensate for the resulting fraction of nonrandom samples.

6.4 A More Detailed Analysis of Fill Rates

For a given number N of urns, a given fill rate $0 < f \leq 1$, and a given level of confidence $0 < p < 1$, we would like to determine the minimum number $r = M(N, f, p)$ of rings necessary so that, among the N urns, the event that at least fN are filled has probability at least p . For $f = 1$, this is the Coupon Collector’s Problem.

For our application, the numbers $M(N, 1, p)$ are too high for practical implementation unless p is taken so small as to seriously impact the robust design features we seek. So, we will prefer to keep the confidence p close to one while decreasing the fill rate f and we will present a rigorous postprocessing strategy in Section 7 to recover full confidence in the quality of the bits generated by the overall design.

Unfortunately, there is no known closed form expression for $M(N, f, p)$. But, the following related function is well-studied: Let $P(N, r, f)$ denote the probability that at least fN out of N urns are filled with exactly r rings. Then,

$$M(N, f, p) = \min\{r : P(N, r, f) \geq p\}$$

and these values can be quickly generated by a computer algebra system.

Let us rephrase this last problem in the language of surjective functions. There are N^r functions from an r -set to an N -set. This is the total number of ways to associate each ring in our design to one of the N urns. (Of course, with variability in periods, there is a nonzero probability that some ring will not be in transition in a given interval and also a chance that some ring will exhibit two transitions in a given interval; we take these as negligible.) The number of surjections (onto functions) among these N^r is given by inclusion-exclusion:

$$J(r, N) := \sum_{h=0}^N (-1)^h \binom{N}{h} (N-h)^r.$$

To prove this, simply note that each function $f : [r] \rightarrow [N]$ is a surjection onto some nonempty subset of $[N] := \{1, 2, \dots, N\}$ and that this is a partition of all functions into surjections. It immediately follows that the number of functions $f : [r] \rightarrow [N]$ with an image of size at least fN is

$$\sum_{k=\lceil fN \rceil}^N \binom{N}{k} J(r, k).$$

The probability that the image has size at least fN is then this value divided by N^r , the total number of functions. This gives us our desired value, $P(N, r, f)$.

In Table 3, we give $M(N, f, p)$ for $N = 36$ urns. (See Table 4 and Table 5 for similar tables for $N = 52$ and 100 .) For each value of N , for each r and f , we compute the probability $P(N, r, f)$ and use a reverse lookup table to obtain the values given in Tables 3, 4, and 5.

7 RESILIENT FUNCTIONS

Even in our model with identical ring lengths, in order to keep the number of rings within a practical range, we have been forced to compromise on the number of filled urns. A

TABLE 3

For the Case $N = 36$ Urns, the Number r of Rings Necessary to Fill at Least fN of the Urns with Probability at Least p Is Shown

p	f									
	.50	.55	.60	.65	.70	.75	.80	.85	.90	.95
.50	26	30	35	41	48	52	61	74	94	142
.55	27	31	36	42	49	53	62	76	96	147
.60	27	31	37	42	50	54	64	77	99	153
.65	27	32	37	43	51	55	65	79	101	159
.70	28	33	38	44	52	56	66	81	104	165
.75	29	33	39	45	53	57	68	83	108	173
.80	29	34	40	46	54	59	70	86	111	182
.85	30	35	41	48	56	61	72	89	116	193
.90	31	36	42	49	58	63	75	93	122	208
.95	33	38	45	52	61	67	80	99	132	233
.99	36	42	50	58	68	75	90	113	153	291

sampler which merely measures the signal at regular intervals and turns these readings into zeros and ones will have, say, 90 percent random bits (each with at least 25 percent chance of being a zero and at least 25 percent chance of being a one) and 10 percent of the samples will hit every ring in a “flat range.” That is, these few samples will be deterministic functions of time. But, we don’t know which are the random ones and which are deterministic. In what follows, we based our model on the assumption that the random bits behave as unbiased coin flips, while being aware that this is a nontrivial simplification.

While this level of randomness may be satisfactory for some applications, we seek to eliminate these nonrandom components to our output by boiling down the bitstream to a shorter, more random sequence of bits. We accomplish this using resilient functions.

Definition 1. An (n, m, t) -resilient function is a function

$$F(x_1, x_2, \dots, x_n) = (y_1, y_2, \dots, y_m)$$

from $\mathbf{Z}_2^n$ to $\mathbf{Z}_2^m$ enjoying the property that, for any t coordinates $i_1, \dots, i_t$, for any constants $a_1, \dots, a_t$ from $\mathbf{Z}_2$ and any element y of the codomain

$$\text{Prob}[F(x) = y \mid x_{i_1} = a_1, \dots, x_{i_t} = a_t] = \frac{1}{2^m}.$$

In the computation of this probability, all x_i for $i \notin \{i_1, \dots, i_t\}$ are viewed as independent random variables, each of which takes on the value 0 or 1 with probability 0.5.

TABLE 4

For the Case $N = 52$ Urns, the Number n of Rings Necessary to Fill at Least fN of the Urns with Probability at Least p Is Shown

p	f									
	.50	.55	.60	.65	.70	.75	.80	.85	.90	.95
.50	37	44	51	57	66	74	88	107	126	179
.55	38	44	52	57	67	75	89	109	128	183
.60	38	45	53	58	68	76	91	111	131	188
.65	39	46	53	59	69	78	93	113	134	194
.70	39	46	54	60	71	79	94	116	137	199
.75	40	47	55	61	72	80	96	118	140	206
.80	41	48	56	63	74	82	99	121	144	213
.85	42	49	58	64	75	84	101	125	149	223
.90	43	51	59	66	78	87	105	130	155	235
.95	45	53	62	69	81	91	110	137	164	255
.99	49	57	68	75	89	100	121	152	185	299

TABLE 5

For the Case $N = 100$ Urns, the Number n of Rings Necessary to Fill at Least fN of the Urns with Probability at Least p Is Shown

p	f									
	.50	.55	.60	.65	.70	.75	.80	.85	.90	.95
.50	70	81	93	106	122	140	163	192	234	307
.55	71	82	94	108	123	142	165	195	237	312
.60	72	83	95	109	125	144	167	197	241	317
.65	73	84	96	110	126	145	169	200	244	323
.70	73	85	97	111	128	147	171	203	248	329
.75	74	86	98	113	129	149	174	206	253	336
.80	75	87	100	114	131	152	177	210	258	344
.85	77	88	101	116	134	155	180	214	263	354
.90	78	90	104	119	137	158	185	220	271	366
.95	81	93	107	123	141	164	191	228	283	386
.99	86	99	114	131	151	175	205	246	307	427

In more informal terms, knowledge of any t values of the input to the function does not allow one to make any better than a random guess at the output. Resilient functions are used in a number of cryptographic applications.

In our setting, the input to the resilient function is the sequence of bits generated by the chip as described above. We have argued that the output is composed of a large number (e.g., $0.9L$) of random bits infiltrated by a small number (e.g., $0.1L$) of bits which are deterministic functions of the circuit. If we feed these L bits into an $(L, m, L/10)$ resilient function, the output must necessarily consist of m truly random bits.

We now show how resilient functions may be obtained from error-correcting codes. A simple technique for constructing resilient functions is given in the following theorem:

Theorem 1 (e.g., [15]). Let G be a generator matrix for an $[n, m, d]$ linear code C . Define a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ by the rule $f(x) = xG^T$. Then, f is an $(n, m, d - 1)$ -resilient function.

For more information on resilient functions, and their connections to codes and designs, see [9] and [10].

Equipped with a construction for resilient functions, we can choose from two approaches in our TRNG design:

- In the urn model, since we do not have control over which urns will be filled by the jitter from any given ring, we may overcompensate by adding a large number of extra rings so that we have a reasonable chance of filling out the spectrum. From the above discussion of the coupon collector problem, we need to use $N \log_2 N$ rings to fill N urns. To be precise, N denotes the ratio between a clock period and the standard deviation of the transition at the edges (i.e., the effective jitter width).
- Alternatively, we can use a reduced number of rings and have only a portion of the spectrum filled with jitter. This means that we will occasionally sample from the deterministic portions of the signal. To filter out these deterministic bits, we use a resilient function with appropriate parameters. The output of the resilient function will give us purely random bits with high probability.

The second approach is clearly preferable since it drastically reduces the number of rings we need to use in our design. On the other hand, we now have to implement a resilient function. Note that we do not necessarily need to build the resilient function in hardware. Since it is a purely deterministic function, it may be easily implemented in software as a postprocessing step. On the other hand, if one wishes to implement the resilient function in hardware, this can still be achieved efficiently. All one has to do is to implement a vector times a (constant) matrix product, as described in Theorem 1. Hence, the presented method does *not* require full error decoding, but is, rather, akin to the calculation of a syndrome. Moreover, if a cyclic code is used to construct the resilient function, one may use a simple linear feedback shift register (LFSR) to obtain an even more compact implementation. Since such constructions are well-known and published, we do not go into the implementation details any further.

There is a trade-off between code length and the size of the buffers we need to use in the implementation of the resilient function. A code of short length is easy to implement and requires smaller buffers, but using such a code runs a higher risk of being compromised when there is a burst of errors created by natural causes or by an intelligent attacker. For example, a code with minimum distance $d = n/2$ used with 80 percent fill rate has a per-block failure rate at most 7×10^{-52} for $n = 1,024$. But, a code with $n = 8$ will fail roughly 13.2 percent of the time with the same input stream, giving a 4-bit sequence of entirely deterministic output. (The 80 percent fill rate on a long input stream is modeled here as a 0.2 per bit probability of determinism so that the resilient function fails on an n -bit input block with probability $\sum_{k=n/2}^n \binom{n}{k} 0.2^k$.)

At this point, we introduce a very simple code that is suitable for our application.

Definition 2. The simplex code $H_m^\perp$, the dual of the Hamming code H_m , is a $[2^m - 1, m, 2^{m-1}]$ linear code.

Note that the minimum distance and the resilience of a resilient function built from a simplex code is roughly half of the block length. Although the output length is logarithmically related to the length of the input, this provides a class of resilient functions with particularly strong resistance against faults.

8 ENTROPY ANALYSIS

In this section, we present an entropy estimate based on the analysis in Section 6.4 and we examine the effect of the resilient function on this estimate.

We first analyze the output entropy without the resilient function. Assuming the fill rate f is achieved with probability at least p and, for purposes of simplification, assuming the output is all zeros when the fill rate is not achieved, we obtain the following probabilities q_a of observing each N bit string a :⁴

$$q_a = \begin{cases} p \cdot 2^{-fN} + (1-p), & \text{if } a = 0 \\ p \cdot 2^{-fN}, & \text{if } a_i = 0 \text{ for } i > fN \text{ and } a \neq 0 \\ 0, & \text{otherwise.} \end{cases}$$

Now, writing $w = p \cdot 2^{-fN} + (1-p)$, we compute the Shannon entropy as follows:

$$\begin{aligned} H &= - \sum_a q_a \log_2(q_a) \\ &= -w \log_2(w) - (2^{fN} - 1) \cdot p \cdot 2^{-fN} (-fN + \log_2(p)) \\ &= p(1 - 2^{-fN}) \log_2(w) - \log_2(w) + \\ &\quad \left(\frac{2^{fN} - 1}{2^{fN}} \right) \cdot p \cdot (fN - \log_2(p)) \\ &\approx pfN \end{aligned}$$

for large fN and $p > 0.5$. Now, we consider the output entropy of the TRNG with the resilient function. Suppose the fill rate exceeds the threshold of the resilient function, i.e., $f > \frac{N-d}{N}$ with probability p . Then, p of the time, the output is perfectly uniform. In the worst case, the output is all zeros with probability $1-p$. The entropy is bounded below by

$$\begin{aligned} H &= \sum_{a \in \{0,1\}^*} -q_a \log_2(q_a) \\ &\geq -(2^m - 1) \cdot p \cdot 2^{-m} \cdot \log_2(2^{-m}) - w \log_2(w), \end{aligned}$$

where $w = q_0 = p \cdot 2^{-m} + (1-p)$. Set $\sigma = p^{\frac{2^m-1}{2^m}}$. Then, since $w \log_2(w) < 0$,

$$H \geq m\sigma \approx mp.$$

9 TOLERANCE AGAINST ATTACKS

We only consider fault-introduction attacks since they are significantly more powerful and more easily implemented on true random number generators than bit-deduction attacks. Simply stated, one can say that it is sufficient to introduce a bias in the output of a TRNG for an attack to be considered successful. We analyze the tolerance of our design against attacks in two categories:

Noninvasive attacks. The types of attacks we consider under this category have to do with *external* influences that an attacker might employ, including, but not limited to, improperly biasing input and/or output bits (e.g., introducing spikes in the power supply), applying electromagnetic shocks to the chip, forcing abrupt temperature changes, etc. For most of the attacks in this category, the duration of the adversaries' influence is limited. Since our design is stateless, only a finite number of bits at the output are influenced. This allows the design to eliminate the *corrupted* bits by simply using a stronger resilient function (i.e., employ an error-correcting code with larger minimum distance).

Invasive attacks. Attacks of this type are more demanding of resources from the attacker's point of view. However, invasive attacks are more powerful and usually have a permanent effect on a circuit. For instance, an attacker may insert a pin (or burn a hole by using a strong laser) into the chip and thereby introduce permanent defects, e.g., cause a short or open circuit by taking out a transistor. Since the

4. Note that, for notational convenience, we are assuming that the unfilled urns are the last ones and a sample of an unfilled urn returns a zero. The analysis will be identical to that which assumes $(1-f)N$ arbitrary deterministic bits distributed arbitrarily across the window.

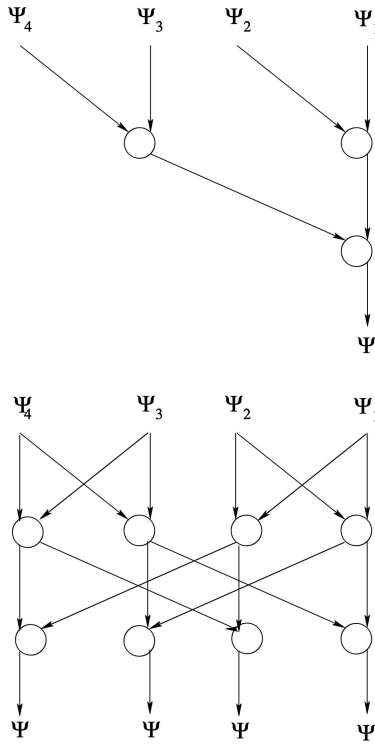


Fig. 5. The original binary tree (top) and the redundant tree (bottom).

attacker's goal is to damage the TRNG and thereby bias its output bits, his job is easy—complex under-the-microscope work that requires high precision is not necessary for him.

To counter such powerful attacks, we need to analyze our design in parts and make sure all parts display the same level of resistance.

- **Oscillator Rings:** An attacker may introduce a permanent fault by taking out one or more oscillators, thereby turning one or more of the ring outputs into deterministic/constant signals. As in the case of noninvasive attacks, our design can easily handle a number of such faults determined by increasing the strength of the resilient function, i.e., by a certain amount of overengineering in the original design.
- **XOR-tree:** We assume that, in the realization of the design, an XOR-tree is used to hash the ring outputs into a single bit which is later sampled. The intuition behind using such a structure is that, as long as any one of the input bits is random, the output of the tree will be random. A 4 bit version of a binary XOR-tree is shown on the top in Fig. 5. In the first level, the ring outputs are XORed together in a pairwise manner. If any single one of these XOR gates are faulted by the adversary, fixing its output to either a zero or one output bits, the effect may be seen as two of the ring outputs having become deterministic and, as before, the bias will be eliminated by the resilient function as much as the built-in strength parameter t permits. If any single XOR-gate in the second level is faulted, the effect is stronger since now the output of that XOR gate comes from four rings outputs. The effect of such faults clearly

doubles at each level and reaches its optimum (as a threat to performance) at the last level, where there is a single XOR-gate. This is a serious problem since now we have a much weaker spot in the design than the rest of the circuit. One way to get around this problem is to simply replicate the XOR-tree and generate the output bit multiple times to meet any desired fault tolerance criterion, but this is a highly wasteful method. For example, a single XOR-tree to hash outputs from r rings will require r XOR-gates (assuming r is a power of two). The direct r -fold replication would require r^2 gates to provide the same level of fault tolerance at the last level as at the top.

To alleviate this problem, we propose replacing the binary XOR-tree with the expanded XOR trellis shown on the bottom of Fig. 5. As is easily seen from the diagram, each XOR-gate now contributes the same level entropy. However, since we still need a single stream of output bits, we need to change the sampling mechanism. That is, the output lines should be sampled one after another in a sequential scheme. Hence, each output line is sampled once in each clock cycle. When compared to the original scheme, the sampling rate remains the same. Only the location of the sampling is changed.

We briefly remark that the trellis depicted is easily scalable. For $r = 2^k$, we have one node for each pair (a, i) , where a is a binary k -tuple and $0 \leq i \leq k$. Node $(a, 0)$ corresponds to the signal Ψ_a and there is a wire from node (a, i) to $(b, i + 1)$ when the binary k -tuples a and b agree in all but possibly the i th position.

- **Sampler:** The sampler for the original binary XOR-tree circuit output would be implemented as a simple D-flip-flop with its clock input set to a signal switching at the sampling frequency. The problem with this approach is, as before, we have a single point of failure. Since we want to protect the circuit from invasive attacks, the only way to get around this problem is to sample all output lines in the expanded tree by simultaneously using r flip-flops and then connecting their output lines to the chip pins. Consequently, the number of output lines may become too excessive even for a moderate number of rings. In this case, one may employ a strategy where a pruned expanded trellis is employed which would have a reduced number of output lines. Such a circuit can be easily constructed by removing some of the output lines from the trellis in Fig. 5 as well as the subcircuits feeding them in the diagram. In the resulting design, the weakest gates in the circuit will again be those gates at the output stage.
- **Resilient function:** If the resilient function is implemented as part of postprocessing in software, then there is no problem since we assume that the postprocessing step is performed on a trusted system. Otherwise, similar precautions need to be taken to implement the resilient function as well. Since the resilient function, in general, will have an irregular

structure compared to the XOR-tree, making it tolerant to invasive attacks is more of an ad hoc process. For example, one may use redundant logic expressions (by avoiding logic minimization) as a means of making it fault-tolerant. On the other hand, if cyclic codes are employed, the resilient function may be realized by using an LFSR circuit. If this is the case, taking advantage of the regularity, one may easily replicate the storage elements to achieve fault tolerance.

10 PUTTING IT ALL TOGETHER—A SAMPLE DESIGN

In this section, we develop a sample design by picking realistic parameters. We quantify the entropy rate and the throughput of the random bit stream generated by our circuit.

If our rings use 13 inverters, then experimental evidence shows that the period is roughly 25 ns and the standard deviation for the jitter random variable is $\sigma = 0.5$ ns [2]. So, $\sigma = 0.02T$. Now, we want all samples to be within $\frac{1}{4}\sigma$ of the mean of some jitter event. So, with tolerance $(\mu - \frac{1}{4}\sigma, \mu + \frac{1}{4}\sigma)$, we can say that 1 percent of the spectrum is filled with jitter for each ring. So, we have $N = 100$ urns in our combinatorial model. We note that the tolerance $\frac{1}{4}\sigma$ ensures that each generated bit will yield at least 0.97 bits of entropy per sampled bit.

With these parameters, the formulae developed in Section 6.4 (or last row, third entry in Table 5) tell us that $r = 114$ rings will be enough to fill at least $0.60N$ of the urns with probability at least 0.99. The output is sampled from the 114 rings and fed into a resilient function. Now, for our resilient function, we employ a [256, 16, 113]-code which is a known extended BCH code [14].⁵ This means that the samples are grouped into blocks of 256 bits and fed into the resilient function which returns only 16 bits. The code we selected has minimum distance $d = 113$ and, therefore, the resilience of the associated resilient function is $t = 112$. With a block length of 256 and fill rate of 0.60, out of the 256 bits which go into our resilient function, $(1 - 0.60) \cdot 256 = 102.4 \approx 103$ bits will be deterministic. Since our resilient function can tolerate up to 112 corrupted bits, the design has an additional margin to resist additional (adversarial and nonadversarial) faults and errors of up to 9 bits. The error margin or tolerance may be increased by either increasing the number of rings or by using an error-correcting code with larger minimum distance.

The output is 16 bits per each 256 bits sampled. These 16 bits each have 0.97 bits of entropy. Since the frequency of the circuit is $\frac{1}{25ns} = 40$ Mhz, this model gives us a random stream with bit rate of $\frac{16}{256} 40 = 2.5$ Mbps, where each bit carries 0.97 bits of entropy. Thus, with modest parameters, we obtain a rather fast random bit generator with provably secure behavior.

5. Although there are many choices for the code we use as our resilient function as discussed above, we choose here a good cyclic code which is not a simplex code.

11 CONCLUSION

In this work, we have outlined the design of a true random number generator which can be built using purely digital logic. We developed design principles that lead to a small footprint design. The elements are simple to analyze and implement. At the same time, our scheme allows one to increase fault tolerance of the random bit generator against active adversaries to any desired level. One rare feature of the design is that it is given in a framework which allows one to exactly quantify the performance and the degree of resilience of the design. We identify further experimentation, especially across a variety of reconfigurable (FPGA, CPLD) and ASIC platforms, as future work.

Finally, we would like to note that, at the last stages of the publication of this paper, we have received notice of [21], which gives details of the realization of our design on a common FPGA platform.

ACKNOWLEDGMENTS

B. Sunar would like to acknowledge support provided by the US National Science Foundation (NSF) through the CAREER Award: New Directions for Cryptographic Hardware, NSF-ANI-0133297. D.R. Stinson's work was supported by NSERC discovery grant 203114-06.

REFERENCES

- [1] B. Barak, R. Shaltiel, and E. Tomer, "True Random Number Generators Secure in a Changing Environment," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '03)*, Ç.K. Koç and C. Paar, eds., pp. 166-180, 2003.
- [2] B. Abcunas, S.P. Coughlin, G. Pedro, and D.C. Reisberg, "Evaluation of RNGs Using FPGAs," Worcester Polytechnic Inst., Major Qualifying Project Report, May 2004.
- [3] W.R. Coppock and C.R. Philbrook, "A Mathematical and Physical Analysis of Circuit Jitter with Application to Cryptographic Random Bit Generation," Worcester Polytechnic Inst., Major Qualifying Project Report, Apr. 2005.
- [4] B. Jun and P. Kocher, "The Intel Random Number Generator," white paper prepared for Intel Corp., Apr. 1999.
- [5] R.A. Schulz, "Random Number Generator Circuit," US Patent Number 4905176, Feb. 1990.
- [6] V. Bagini and M. Bucci, "A Design of Reliable True Random Number Generator for Cryptographic Applications," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '99)*, Ç.K. Koç and C. Paar, eds., pp. 204-218, 1999.
- [7] V. Fischer and M. Drutarovský, "True Random Number Generator Embedded in Reconfigurable Hardware," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '02)*, B.S. Kaliski Jr., Ç.K. Koç, and C. Paar, eds., pp. 415-430, 2002.
- [8] T.E. Tkacik, "A Hardware Random Number Generator," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '02)*, B.S. Kaliski Jr., Ç.K. Koç, and C. Paar, eds., pp. 450-453, 2002.
- [9] C.J. Colbourn, J.H. Dinitz, and D.R. Stinson, "Applications of Combinatorial Designs to Communications, Cryptography and Networking," *Surveys in Combinatorics (Proc. 1999 British Combinatorial Conf.)*, pp. 37-100, 1999.
- [10] D.R. Stinson and K. Gopalakrishnan, "Applications of Designs to Cryptography," *CRC Handbook of Combinatorial Designs*, C.D. Colbourn, and J.H. Dinitz, eds., CRC Press, 1996.
- [11] M. Epstein, L. Hars, R. Krasinski, M. Rosner, and H. Zheng, "Design and Implementation of a True Random Number Generator Based on Digital Circuit Artifacts," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '03)*, C.D. Walter, Ç.K. Koç, and C. Paar, eds., pp. 152-165, 2003.
- [12] G. Marsaglia, *DIEHARD: A Battery of Tests of Randomness*, <http://stat.fsu.edu/~geo>, 1996.

- [13] NIST Special Publication 800-22, "A Statistical Test Suite for Random and Pseudorandom Numbers," Dec. 2000.
- [14] A.E. Brouwer, "Server for Bounds on the Minimum Distance of q -ary Linear Codes, $q = 2, 3, 4, 5, 7, 8, 9$," <http://www.win.tue.nl/~aeb/voorlincod.html>.
- [15] B. Chor, O. Goldreich, J. Håstad, J. Friedman, S. Rudich, and R. Smolensky, "The Bit Extraction Problem or t -Resilient Functions," *Proc. 26th IEEE Symp. Foundations of Computer Science*, pp. 396-407, 1985.
- [16] M. Dichtl, "How to Predict the Output of a Hardware Random Number Generator," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '03)*, C.D. Walter, Ç.K. Koç, and C. Paar, eds., pp. 181-188, 2003.
- [17] W. Schindler, "A Stochastic Model and Its Analysis for a Physical Random Number Generator," *Proc. Cryptography and Coding '03*, K.G. Paterson, ed., pp. 276-289, 2003.
- [18] W. Schindler and W. Killmann, "Evaluation Criteria for True (Physical) Random Number Generators Used in Cryptographic Applications," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '02)*, B.S. Kaliski Jr., Ç.K. Koç, and C. Paar, eds., pp. 431-449, Aug. 2002.
- [19] M. Bucci and R. Luzzi, "Design of Testable Random Bit Generators," *Proc. Workshop Cryptographic Hardware and Embedded Systems (CHES '05)*, J.R. Rao and B. Sunar, eds., pp. 131-146, Aug. 2005.
- [20] Anwendungshinweise und Interpretationen zum Schema (AIS), AIS 32, Version 1, Bundesamt für Sicherheit in der Informationstechnik, 2001.
- [21] D. Schellekens, B. Preneel, and I. Verbauwhede, "FPGA Vendor Agnostic True Random Number Generator," *Proc. 16th Int'l Conf. Field Programmable Logic and Applications*, to appear.



Berk Sunar received the BSc degree in electrical and electronics engineering from Middle East Technical University in 1995 and the PhD degree in electrical and computer engineering (ECE) from Oregon State University in December 1998. After briefly working as a member of the research faculty at Oregon State University, he joined Worcester Polytechnic Institute as an assistant professor. Since July 2006, he has been an associate professor. He currently heads

the Cryptography and Information Security Laboratory (CRIS). He received the US National Science Foundation CAREER award in 2002. He organized the Cryptographic Hardware and Embedded Systems Conference (CHES) in 2004 and is the coeditor of CHES 2005. His research interests include finite fields, elliptic curve cryptography, low-power cryptography, and computer arithmetic. He is a member of the IEEE Computer Society, the ACM, and the International Association of Cryptologic Research (IACR).



Applied Cryptography at the University of Waterloo and the Department of Mathematics at the Massachusetts Institute of Technology.



Douglas R. Stinson received the BMath degree from the University of Waterloo, Canada, in 1978, the MSc degree from Ohio State University in 1980, and the PhD degree in combinatorics and optimization from the University of Waterloo in 1981. He previously has held academic positions at the University of Manitoba, where he was an NSERC University Research Fellow, and the University of Nebraska-Lincoln. Currently, he holds the position of professor and University Research Chair in the David R. Cheriton School of Computer Science at the University of Waterloo. He held the NSERC/Certicom Industrial Research Chair in Cryptography from 1998-2003. His research interests include cryptography and computer security, combinatorics and coding theory, and applications of discrete mathematics in computer science. He is the author of more than 200 research papers as well as the popular textbook *Cryptography: Theory and Practice*, the third edition of which was published in 2005. He was one of the founding coeditors of the *Journal of Combinatorial Designs* and he has served on the editorial boards of numerous other journals. He is currently the chair of the Selected Areas in Cryptography (SAC) Organizing Board, a leading cryptography workshop that has been held annually in Canada since 1994.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.